

BOARD OF INTERMEDIATE EDUCATION KARACHI

HIGHER SECONDARY · COMPUTER SCIENCE

Class XII

Lecture notes Computer Science

Sindh Curriculum 2024 · National Curriculum of Pakistan 2022–23

Original lecture notes for classroom and self-study

Session 2026–27

How to use these lectures

These lectures are written for Higher Secondary students taking Computer Science with the Board of Intermediate Education Karachi. They follow the Sindh Curriculum of Computer Science 2024, which is aligned with the National Curriculum of Pakistan 2022–23 and with the Sindh Textbook Board books for the 2026–27 session. Each lecture is one teaching block: the idea, a worked example, and a short checkpoint in the style of the HSSC paper. Python programs that are complete scripts show the output produced by running them on Python 3.

What each lecture contains

- **Learning outcomes** match the student learning outcomes of the National Curriculum of Pakistan 2022–23 for this grade.
- **Explanations and worked examples** are written so that a student can revise a period without a second textbook open.
- **Python programs** that are complete scripts include the output produced by running them.
- **Checkpoint** questions follow the HSSC pattern: one-mark choices and three-mark short answers, with a key underneath.

A note on the source of the course

Karachi intermediate colleges teach the Sindh Textbook Board books. For 2026–27 those books follow the Sindh Curriculum of Computer Science 2024, which is aligned with the National Curriculum of Pakistan 2022–23. Class XI and Class XII are organised by the same domains: Computer Systems, Computational Thinking and Algorithms, Programming Fundamentals, Data and Analysis, Applications and Impacts of Computing, Digital Literacy, and Entrepreneurship in the Digital Age. Programming in this course is Python, not the older C++ syllabus.

REMEMBER

These notes are an original teaching set. They are not an official BIEK paper, not a past paper, and not a copy of the Sindh Textbook Board book. Use the textbook alongside them for exercises set by your teacher.

Units in this booklet

Unit	Title	Emphasis in the course
Unit 1	Computer Systems	About 30 periods
Unit 2	Computational Thinking and Algorithms	About 20 periods
Unit 3	Programming Fundamentals	About 40 periods
Unit 4	Data and Analysis	About 25 periods
Unit 5	Applications and Impacts of Computing	About 30 periods
Unit 6	Digital Research and Entrepreneurship	About 20 periods

Contents

Unit 1 Computer Systems	4
Lecture 1.1 Usability, accessibility, and HCI	4
Lecture 1.2 Security, usability, and the tradeoff	5
Unit 2 Computational Thinking and Algorithms	7
Lecture 2.1 Judging a solution	7
Lecture 2.2 Lists, stacks, queues, and trees	8
Lecture 2.3 Traversals, binary search, and breadth versus depth	9
Unit 3 Programming Fundamentals	11
Lecture 3.1 Paradigms, classes, and objects	11
Lecture 3.2 Lists, dictionaries, and nesting	13
Lecture 3.3 Text files	14
Lecture 3.4 SQL and third normal form	15
Lecture 3.5 Testing and debugging	17
Unit 4 Data and Analysis	18
Lecture 4.1 Learning from data, and knowing if it worked	19
Lecture 4.2 Visualisation that makes a claim	20
Lecture 4.3 Hypothesis testing	21
Unit 5 Applications and Impacts of Computing	22
Lecture 5.1 Designs for Pakistan: IoT, cloud, and shared records	22
Lecture 5.2 Deep learning, privacy, and policy	23
Lecture 5.3 Threats, controls, and equal access	25
Unit 6 Digital Research and Entrepreneurship	26
Lecture 6.1 A research artefact	26
Lecture 6.2 From a prototype to a minimum viable product	27

Unit 1 Computer Systems

Class XI treated the machine, the network, and the project plan. Class XII asks a different question about the same systems: can a person actually use them, and what is given up when they are made safer? The answers are practical. A secure screen that nobody can operate is not a successful system.

OUTCOMES

By the end of this unit you should be able to: Explain usability, accessibility, and a basic usability test. Describe the tradeoff between security and ease of use, and name controls that respect both.

Lecture 1.1 Usability, accessibility, and HCI

OUTCOMES

Define usability and accessibility in terms of a task a person is trying to finish. Describe a usability test and the kinds of fault it finds.

Human-computer interaction

Human-computer interaction (HCI) is the study of how people get a job done with a computer system. It is not a chapter about making screens fashionable. A system is **usable** when the intended person can finish the right task, in a reasonable time, without a preventable mistake, and without feeling lost. Those four points are effectiveness, efficiency, safety of the action, and satisfaction. A portal that eventually shows the date sheet, after seven unlabelled menus, has failed efficiency even though the date sheet exists somewhere.

Accessibility means people with disabilities, and many older users, can finish the same tasks. A screen reader must be able to read the labels. A control must not depend on colour alone, because some users cannot distinguish red from green. Text must still make sense when it is enlarged. Captions must exist where the content is speech. An inaccessible system does not become acceptable because most of the class can use it. The students who cannot use it are still students.

Faults you should be able to name

- Tiny text and low contrast. Grey words on a grey button cannot be read in sunlight or by a user with low vision.
- Labels that assume knowledge. A button called 'Submit' does not say whether it searches or sends.
- Too many steps for a frequent task. The exam date should not be buried.
- No confirmation of a destructive action. Deleting a practical file with one unmarked tap is a usability fault and a data fault.
- Messages that blame the user. 'Invalid input' does not say which field, or what a valid value looks like.

Usability testing

A **usability test** gives a realistic task to a person who did not build the system, and watches them try. You do not demonstrate the screen first. You do not say 'the button is on the left' while they are stuck. You record where they pause, what they click, and whether they finish. Five people will usually reveal the disasters. A test is not a survey that asks 'do you like the blue?'. Liking a colour and finishing a task are different measurements.

Improvements follow the notes. Put the frequent action on the first screen. Use words from the user's language, including Urdu where the users think in Urdu. Show the focus of the keyboard. Repeat a control in the same place on every screen. After the change, test again with a new person. The designer is a poor test user, because the designer already knows the path.

Consequences beyond the screen

A bad design has effects the syllabus asks you to separate. **Ethical:** people are excluded or misled. **Social:** a family member has to operate the service for someone who could have operated it. **Economic:** staff spend the day answering questions the screen should have answered, and users abandon a service. **Environmental:** confused users repeat

downloads and print screens they cannot read. None of these is fixed by a new logo.

EXAM TIP

A five-mark HCI question often wants five headings: usability, a common problem, a method of improvement, an ethical or social effect, and an economic or environmental effect. Use the headings. One paragraph that mixes them is harder to mark.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

- Which action is a usability test?
 - Asking friends whether the logo is attractive
 - Watching a new user try to find tomorrow's room, without helping them
 - Counting the lines of code
 - Encrypting the password file
- Define usability and accessibility so that the difference is clear. One sentence each.
- Give one common interface problem, one method of improvement, and one social effect of leaving the problem in place.

ANSWERS

- B. A usability test observes a task. Opinion about a logo is not evidence that the task can be finished.
- Usability is how effectively and comfortably the intended users finish their tasks. Accessibility is whether people with disabilities can finish those same tasks.
- Low-contrast text is a common problem. A usability test followed by darker text on a light background is a method of improvement. Left unchanged, a student with low vision must ask someone else to read the date sheet, which is a social cost.

Lecture 1.2 Security, usability, and the tradeoff

OUTCOMES

Explain a real tradeoff between making a system safer and making it easier. Recommend a control that considers cost, privacy, and the user's task.

Both goals are real

A system that anyone can open is easy and unsafe. A system that demands a new 20-character password every day, with no recovery path, will be defeated by the sticky note on the monitor. The password rule did not fail in theory. It failed as HCI. **Security** and **usability** are both requirements. When they pull apart, the design has to say what was traded and who bears it.

Choice	What it gains	What it costs
Very short passwords	People remember them	They are easy to guess
Long unique passwords plus a manager	Hard to guess, not reused	The user must learn the manager; losing the master password is serious
2FA with a phone code	A stolen password is not enough	A user who has lost the phone is locked out
Biometric unlock	Fast, nothing to remember	A fingerprint cannot be changed if the template leaks; some people cannot use the sensor
Logging every click 'for safety'	An investigation has evidence	Students are watched far beyond the threat

Table. A good answer names both columns. A control with no cost has not been thought through.

Deciding

Four factors keep returning. **Efficiency:** does the control add so many steps that the work moves to a less safe channel, such as a personal chat app? **Cost:** can the college run it, including support when people are locked out? **Privacy:** does the control collect a fingerprint, a location, or a document it does not need? **Ethics:** who is refused service, and is there a human appeal? A design that ignores any one of these will be 'secure' only in a slide.

A **zero-trust** approach means the network location is not treated as proof. A laptop on the college Wi-Fi is still asked to authenticate, and a request for the result database is still checked for authorization. Zero trust is not a product name. It is a refusal to say 'they are inside the building, so they may see every file'.

Usable security is often a better default, not a weaker one. Signing in with a managed single sign-on, so that one protected login opens the portal and the mail, removes the temptation to reuse a weak password on six sites. Clear labels ('we sent a code to the phone ending 14') reduce the calls to the office. Recovery that a real person can complete, without posting the password in a group chat, is part of security.

EXAMPLE

A college blocks all phones from the learning site 'to prevent cheating', including the evening, when many students only have a phone. The security aim is real. The effect is that those students share logins on a friend's computer, which is less safe. A tighter rule is to restrict the exam window, not the whole evening, and to keep ordinary reading available.

REMEMBER

Class XI defined authentication, authorization, 2FA, and encryption. This lecture's new demand is the tradeoff: say what the user loses, not only what the attacker loses.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

- Which situation is a usability-security tradeoff?
 - A longer unique password is harder to guess and harder to remember
 - A loop prints the numbers 1 to 10
 - A primary key must be unique
 - A histogram shows a distribution
- Give three factors to weigh before a college adds a new security control.
- Why can a rule that looks safer on paper make the system less safe in practice? Give one example.

ANSWERS

- A. The same decision improves security and worsens usability.
- Efficiency of the user's real task, the cost including support when people are locked out, and privacy: whether the control collects data it does not need. Ethics of who is excluded belongs with these.
- If the rule is too hard, people route around it. Forcing a new complex password every day leads users to write it on the monitor, so the password is no longer secret.

Practical work

These are observation practicals. Write what the user did, not what you hoped they would do.

- Watch a classmate complete two tasks on the college site or any public site: find a date, and find a contact address. Note every place they hesitate.
- Check the same two screens for a text size that can be enlarged, labels that are not colour-only, and a visible keyboard focus. Write three concrete faults or three things done well.
- For a login screen, propose one change that adds security and one change that protects usability. Say who is helped by each.

Unit 2 Computational Thinking and Algorithms

Class XI searched and sorted lists. Class XII judges those solutions and adds the structures that are not a flat list: stacks, queues, and trees. A tree is how a file system, a family of headings, and a binary search of sorted keys are organised. You will trace searches on them by hand.

OUTCOMES

By the end of this unit you should be able to: Judge an algorithm for correctness, clarity, and efficiency. Trace in-order, pre-order, and post-order walks, and trace binary search. Choose a stack, a queue, or a tree for a described problem.

Lecture 2.1 Judging a solution

OUTCOMES

Use correctness, clarity, and efficiency as three separate judgements. Explain why halving beats scanning once the list is long and sorted.

Three different compliments

An algorithm can be admired for the wrong reason. **Correctness** means that every allowed input produces the output the specification promised, including the awkward cases: an empty list, a missing target, two equal values. **Clarity** means another student can trace it without asking you what a step meant. **Efficiency** means the amount of work grows reasonably as the input grows. A correct algorithm can be unclear. A clear algorithm can be too slow for the size you actually have. Say which of the three you are discussing.

A useful efficiency comparison at this grade counts the dominant step. Linear search does about n comparisons on a list of n items when the target is absent. Binary search does about $\log_2 n$ comparisons, but only on sorted data. Doubling a sorted list adds one comparison to binary search and doubles the work of linear search. That single sentence is the reason a phone book is not searched from the first page.

n, the number of items	Linear search, about	Binary search, about
8	8	3
1,000	1,000	10
1,000,000	1,000,000	20

Table. The binary column is the number of times you can halve n before one item remains. It assumes the data is sorted.

Clarity is not decoration. Naming a marker `low` rather than `x`, and indenting the body of a loop, are part of correctness in practice, because an unclear step is the step that gets implemented wrongly. In a written trace, clarity means the marker shows every swap or every discarded half, not only the final list.

EXAM TIP

If the question says 'how do we assess a computational solution?', do not describe only speed. Give correctness, clarity, and efficiency, with one sentence of meaning each.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

- The main advantage of binary search over linear search on a long sorted list is that
 - it works on unsorted data as well
 - it halves the remaining search space at each step
 - it never compares values
 - its time grows in proportion to n

2. An algorithm always returns the right total, but a second student cannot tell which variable holds the count. Which quality is strong, and which is weak?
3. A sorted list grows from 1,000 items to 2,000. What happens to the worst-case work of linear search, and of binary search?

ANSWERS

1. B. Halving is the advantage, and it requires sorted data. Growing in proportion to n describes linear search.
2. Correctness is strong. Clarity is weak. Efficiency has not been described.
3. Linear search roughly doubles, from about 1,000 comparisons to about 2,000. Binary search adds about one comparison, because one extra halving covers the doubling.

Lecture 2.2 Lists, stacks, queues, and trees

OUTCOMES

State the rule of a stack and the rule of a queue. Define a binary search tree and draw one from a sequence of insertions.

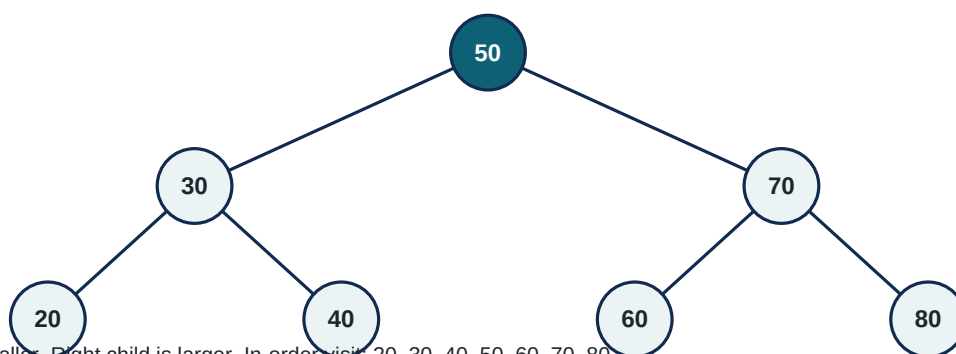
Structures remember a rule

A **data structure** is a way of organising values so that a particular operation is easy and the others are honest about their cost. A list keeps order and supports an index. You already use it in Python. Two further rules appear constantly.

A **stack** is last-in, first-out. The item you put on most recently is the item you take off next. Undo in an editor is a stack: the latest edit is the first one reversed. A **queue** is first-in, first-out. A print queue and a line of students at a window are queues. Using a stack for a print queue would print the latest job first, which is a different policy, not a small coding choice.

Trees

A **tree** stores values in nodes with a parent and children. One node, the **root**, has no parent. A **leaf** has no children. A **binary tree** gives each node at most two children, called left and right. A **binary search tree** adds an order: every value in the left subtree is less than the node, and every value in the right subtree is greater. Duplicates need a stated policy. In this course, do not insert a duplicate; say that it is already present.



Left child is smaller. Right child is larger. In-order visit: 20, 30, 40, 50, 60, 70, 80.

Binary search tree used in the Class XII traversal examples.

The tree in the figure is what you get by inserting 50, then 30, 70, 20, 40, 60, and 80, each time walking left for a smaller value and right for a larger one. The in-order sequence, defined in the next lecture, is the sorted list. That is the point of the structure. A search starts at the root and walks left or right exactly as binary search discards a half, except the 'half' is a subtree.

Trees represent hierarchies that a list represents badly: folders inside folders, headings inside chapters, and the staff chart of a college. The parent of a node is its one step upward. The children are its direct reports, or its subfolders. A

value can appear at most once in the binary search trees you will be asked to draw.

REMEMBER

Left means smaller, right means larger, and this is only for a binary search tree. An ordinary binary tree may place the children in any order the problem states. Read the question before you assume search order.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

- Which structure returns the most recently stored item first?
 - Queue
 - Stack
 - Binary search tree
 - Table
- Insert 15, then 8, then 22, then 10 into an empty binary search tree. State the parent of 10.
- A college printer should print jobs in the order they were sent. Which structure matches, and why is the other linear structure wrong?

ANSWERS

- B. A stack is last-in, first-out.
- 15 is the root. 8 goes left of 15. 22 goes right of 15. 10 is greater than 8 and less than 15, so it goes right of 8. The parent of 10 is 8.
- A queue, because the first job sent is the first job printed. A stack would print the latest job first.

Lecture 2.3 Traversals, binary search, and breadth versus depth

OUTCOMES

Write the in-order, pre-order, and post-order sequences of a given tree. Trace binary search, and contrast a breadth-first walk with a depth-first walk.

Three walks

A **traversal** visits every node once. The name says when the node itself is visited, relative to its subtrees. **In-order:** left subtree, then the node, then the right subtree. **Pre-order:** the node, then left, then right. **Post-order:** left, then right, then the node. Apply the same rule at every node, not only at the root. On a binary search tree, in-order produces the sorted sequence. That is the check you should do after every in-order trace.

For the tree with root 50, left child 30 with children 20 and 40, and right child 70 with children 60 and 80:

- In-order: 20, 30, 40, 50, 60, 70, 80.
- Pre-order: 50, 30, 20, 40, 70, 60, 80.
- Post-order: 20, 40, 30, 60, 80, 70, 50.

If your in-order sequence is not sorted, you have either broken the search-tree property or you have visited a node at the wrong moment. Do not 'fix' the sequence by sorting it afterwards. The sequence is the trace.

Binary search on the sorted list

The in-order list of that tree is 20, 30, 40, 50, 60, 70, 80. Binary search for 60 starts with low at 20 and high at 80. The middle is 50. 60 is larger, so the left half including 50 is discarded. The new middle of 60, 70, 80 is 70. 60 is smaller, so the right side is discarded. The next middle is 60. Found. Searching for 25 ends when low moves past high: 25 is not in the list, and the algorithm must say so rather than return a neighbour.

Depth-first and breadth-first

On a general tree, such as a staff chart, two systematic walks matter. **Depth-first** follows one child as far as it will go before it tries the next sibling. **Breadth-first** visits every node at the present level before it visits the level below. Breadth-first uses a queue. Depth-first uses a stack, or the call stack of recursion.

EXAMPLE

Principal has three departments, in this order: Admin, Security, Accounts. Security has two offices, in this order: Gate and CCTV. Depth-first, taking children left to right, visits Principal, Admin, Security, Gate, CCTV, Accounts. Breadth-first visits Principal, Admin, Security, Accounts, Gate, CCTV. The sequences differ. Name the walk before you write the names.

Use breadth-first when the question is 'who is closest to the root', because the first time you reach a node you reached it by the fewest edges. Use depth-first when you are exploring one branch completely, such as every file inside one folder before the next folder. Both visit every node if you let them finish. They are not searches that know a binary-search order unless the tree is a binary search tree and you use that order.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

1. The in-order traversal of a binary search tree produces
 - A. the insertion order
 - B. the values in sorted order
 - C. only the leaves
 - D. the values in reverse of pre-order, always
2. Using the 50 / 30 / 70 tree from this lecture, give the pre-order sequence and say which node is visited first and why.
3. On the staff chart in this lecture, which walk reaches Accounts before Gate? Give the sequence up to Accounts.

ANSWERS

1. B. Left, node, right respects the smaller-left and larger-right rule, so the values come out sorted.
2. 50, 30, 20, 40, 70, 60, 80. Pre-order visits the node before its subtrees, so the root 50 is first.
3. Breadth-first. Principal, Admin, Security, Accounts. Gate is on the next level, so it comes after Accounts.

Practical work

Traces are the practical. Do them on paper until the order is boringly reliable, then implement one walk in Python if your teacher asks.

- 1 On the tree in Lecture 2.2, write the in-order, pre-order, and post-order sequences without looking at the notes. Check afterwards.
- 2 Insert 45 into that binary search tree and write the new in-order sequence. It should still be sorted.
- 3 Trace binary search for 60 and for 25 on the sorted list 20, 30, 40, 50, 60, 70, 80. For 25, show the step where low passes high.
- 4 Draw a queue of four print jobs and a stack of four undone edits. Show the state after one removal from each.

Unit 3 Programming Fundamentals

Class XII Python is the Class XI language with four additions: a class that gathers data and behaviour, dictionaries and nested lists, files and database tables, and a habit of testing a function on purpose. You still write ordinary functions. A class is a new way to organise them, not a different kind of thinking.

OUTCOMES

By the end of this unit you should be able to: Define a class with a constructor and methods, and explain encapsulation at this grade's level. Use lists, dictionaries, and text files, and write a simple SQL query. Put a table into third normal form and test a function with a deliberate check.

Lecture 3.1 Paradigms, classes, and objects

OUTCOMES

Contrast procedural code with a class, at the level of organisation. Write a Python class with attributes, a constructor, and a method.

Why there is more than one style

A **programming paradigm** is a way of organising a program so that a large problem stays understandable. A **procedural** program is a sequence of functions that pass values around. That is most of Class XI, and it remains the right style for a short calculation. An **object-oriented** program groups the values and the functions that belong to one kind of thing. A **functional** style emphasises functions that compute a result from their inputs and avoid changing values in place. You will write the second of these in Python. You only need to recognise the third.

Object-oriented organisation helps when many pieces of data travel together and several operations apply to them. A vehicle has a type, a brand, a model, and a mileage, and you can update the mileage or display the vehicle. Keeping those four values in four unrelated variables invites a mistake where you update one vehicle and display another. A class makes that mistake harder.

The vocabulary, without the mythology

- A **class** is the definition: the attributes and the methods.
- An **object** is one actual vehicle or one actual student built from that class.
- A **method** is a function defined inside the class. Its first parameter is `self`, the object the method was called on.
- The **constructor** in Python is `__init__`. It runs when the object is created and stores the starting attributes.
- **Encapsulation** means the data and the operations live together, and callers use the methods rather than poking at details they do not own.

Inheritance, overriding, and a full discussion of polymorphism belong to older syllabi and to later study. If a question uses those words, inheritance means a new class that reuses a parent class, and overriding means the new class supplies its own version of a method. You will not be asked to build a deep hierarchy in this course. You will be asked to define one class and use two objects.

```
class Vehicle:
    def __init__(self, vehicle_type, brand, model, mileage):
        self.vehicle_type = vehicle_type
        self.brand = brand
        self.model = model
        self.mileage = mileage

    def update_mileage(self, mileage):
        self.mileage = mileage

    def display(self):
        kind = f"{self.vehicle_type}: {self.brand}"
        return kind + f" {self.model}, {self.mileage} km"

v1 = Vehicle("Car", "Suzuki", "Alto", 12000)
v2 = Vehicle("Motorcycle", "Honda", "CD70", 8000)
v1.update_mileage(12500)
print(v1.display())
print(v2.display())
```

Output

```
Car: Suzuki Alto, 12500 km
Motorcycle: Honda CD70, 8000 km
```

Updating v1 does not change v2. That is the demonstration that each object has its own attributes. `display` returns a string so the caller can print it, test it, or store it. Printing inside the method would be harder to test.

REMEMBER

The two underscores in `__init__` are part of the name. `init` without them is an ordinary method that Python will not call automatically.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

1. What is the main organisational benefit of a class?
 - A. It removes the need for variables
 - B. It keeps related data and operations together
 - C. It always runs faster than a function
 - D. It encrypts the attributes
2. In the Vehicle program, what is the class, what is an object, and what does `update_mileage` change?
3. Why does `display` return the string instead of printing it?

ANSWERS

1. B. A class is an organisational tool. It does not by itself make code faster or secret.
2. Vehicle is the class. v1 and v2 are objects. `update_mileage` changes the mileage of the object it was called on, and not the other object.
3. The caller can then print it, compare it in a test, or write it to a file. A method that only prints cannot hand the text to the next step.

Lecture 3.2 Lists, dictionaries, and nesting

OUTCOMES

Retrieve a value from a dictionary and from a nested list. Choose the structure that matches the lookup the program actually does.

Dictionary

A **list** keeps items in order and finds them by a position. A **dictionary** keeps pairs. The **key** is what you look up. The **value** is what you get back. `student = {"name": "Ali", "grade": 85}` then `student["grade"]` is 85. Looking up a missing key stops the program with a `KeyError` unless you use `.get(key)`, which returns `None` when the key is absent. Keys should be values that do not change, such as strings and numbers. A roll number is a good key. A list is not.

A dictionary answers 'find this student' without walking the whole collection in your own loop. A list answers 'give me the next student in this order' and 'what is in position 3'. When the question is about order, use a list. When the question is about an identity, use a dictionary. You will often use both.

```
students = [
    [101, "Ali", 85],
    [102, "Babar", 90],
    [103, "Sana", 78],
]
by_id = {
    101: {"name": "Ali", "grade": 85},
    102: {"name": "Babar", "grade": 90},
    103: {"name": "Sana", "grade": 78},
}
print(by_id[102]["grade"])
found = None
for row in students:
    if row[0] == 102:
        found = row[2]
print(found)
```

Output

```
90
90
```

Both designs store the same three students. The dictionary retrieves Babar's grade by the roll number directly. The nested list walks the rows until the roll number matches. The nested list is still the right store when you need the class list in a fixed printed order. A dictionary does not promise an order you can rely on for a mark sheet unless you sort the keys yourself.

Nesting

A **nested list** is a list whose items are lists. A table of rows is exactly that. `students[1][2]` is row index 1, column index 2, which is 90 in the list above. A dictionary can hold a list, and a list can hold a dictionary. Go one level at a time when you trace. Write the outer lookup first, then the inner one. Most errors in nested structures are an index applied to the wrong level.

EXAM TIP

A question that shows both structures wants the comparison: what is easy in each. Do not claim that dictionaries are 'more advanced' or always better.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

1. Which lookup is the natural job of a dictionary?
 - A. The item at position 0
 - B. The value stored under a known roll number
 - C. The last row of a table, because it was appended last
 - D. A slice of five neighbouring items
2. In the nested list in this lecture, what is `students[0][1]`, and what is `students[2][0]`?
3. You must print a class in roll-number order and also jump to one student while marking. How can the two structures work together?

ANSWERS

1. B. A dictionary retrieves by key. Position, append order, and slices are list ideas.
2. `students[0][1]` is Ali. `students[2][0]` is 103.
3. Keep a list of roll numbers in the printed order, and a dictionary from roll number to the student's record. Walk the list to print. Use the dictionary when one script arrives out of order.

Lecture 3.3 Text files

OUTCOMES

Explain why a program uses a file. Write a small program that creates a text file and reads it back.

Why the disk

Variables disappear when the program ends. A **file** keeps the bytes after the program has stopped, and another program can read them later. A mark sheet that must still exist tomorrow belongs in a file or in a database, not only in a list. A text file stores characters. You can open it in an editor and see the names. A binary file stores a format that is not plain characters. This course asks you to handle text.

Opening a file says three things: the path, the mode, and the encoding. Mode "w" creates the file or empties it if it already exists. Mode "a" adds to the end. Mode "r" reads. Encoding "utf-8" is the one to name, so that Urdu and ordinary English both have a defined representation. Forgetting that "w" empties the file is how practical journals get destroyed.

```
from pathlib import Path

path = Path("cities.txt")
path.write_text("Karachi\nHyderabad\n", encoding="utf-8")
print(path.read_text(encoding="utf-8"))
```

Output

```
Karachi
Hyderabad
```

The longer form, which shows the open and the close, is this. The with line closes the file even if a later line fails.

```
with open("cities.txt", "w", encoding="utf-8") as handle:
    handle.write("Karachi\n")
    handle.write("Hyderabad\n")

with open("cities.txt", "r", encoding="utf-8") as handle:
    for line in handle:
        print(line.strip())
```

Output

```
Karachi
Hyderabad
```

`strip` removes the newline that `for` line keeps. If you print the line without stripping it, you see a blank line between cities, because `print` adds a newline of its own. That is a logic surprise, not a missing city.

Counting letters is the standard 'read a file and summarise' exercise. Read the text, ignore anything that is not a letter, and use a dictionary whose keys are characters.

```
text = "Banana"
counts = {}
for ch in text.lower():
    if ch.isalpha():
        counts[ch] = counts.get(ch, 0) + 1
print(counts)
```

Output

```
{'b': 1, 'a': 3, 'n': 2}
```

REMEMBER

Always say what "w" does to an existing file. The safe classroom habit is to write to a new name while you are testing.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

- Which mode should you open with when you want to add a line without deleting the lines already in the file?
 - "w"
 - "a"
 - "x" only, there is no other way
 - "r"
- Why is a file needed for a class list that must still be there after the program stops?
- A file contains three names, each written with a newline. A student says the program 'printed blank lines between the names'. What is the likely cause?

ANSWERS

- B. Append adds at the end. Write truncates the existing file.
- Variables exist only while the program is running. A file stores the characters on disk so a later run, or another program, can read them.
- The loop kept the newline from the file and `print` added another. `strip()` on the line removes the file's newline.

Lecture 3.4 SQL and third normal form

OUTCOMES

Write `SELECT` with `WHERE` and a simple aggregate. Explain 1NF, 2NF, and 3NF on one example and split a table that breaks them.

Asking a table a question

SQL is the language used to define and question relational tables. You need a small working subset. `SELECT` chooses columns. `FROM` names the table. `WHERE` keeps some rows. `ORDER BY` sorts the rows that remain. `AVG`, `COUNT`, `SUM`, `MIN`, and `MAX` summarise a column. This is enough for the theory paper and for a practical file.

Take this table, called `employees`.

id	name	department	salary
1	Ayesha	IT	80000

id	name	department	salary
2	Bilal	HR	60000
3	Hina	IT	100000

Table. Salaries are yearly rupees in this example.

`SELECT * FROM employees WHERE department = 'IT';` returns Ayesha's row and Hina's row. The star means every column. The quotes around IT matter: department is text.

`SELECT AVG(salary) AS average_salary FROM employees;` returns 80000, because $(80000 + 60000 + 100000) / 3 = 80000$. The name after AS is the heading of the result, not a new column stored in the table.

`SELECT name, salary FROM employees WHERE department = 'IT' ORDER BY salary;` returns Ayesha then Hina, and it does not return the department column, because the select list did not ask for it. Choosing columns is called projection. Choosing rows is called selection. Use those two words if the question uses them.

Normal form

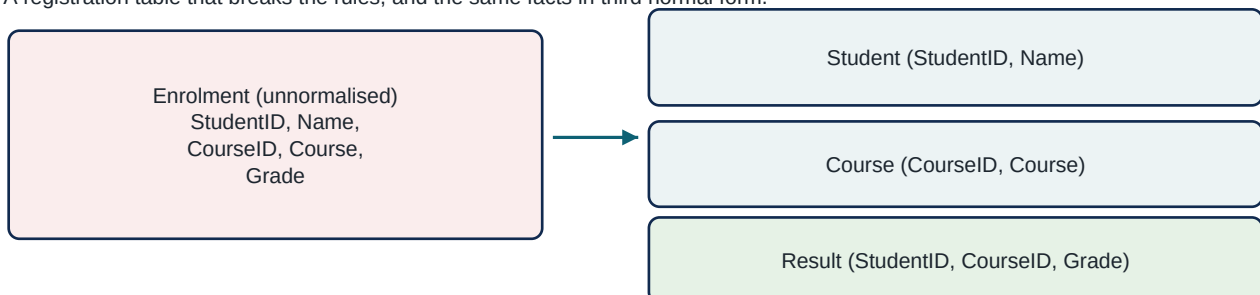
Normal form is a discipline that removes repetition which can disagree with itself. Work in order.

First normal form. Every field holds one value, not a list. A cell that contains two phone numbers breaks 1NF. Split the numbers into their own rows.

Second normal form. The table is in 1NF, and every non-key field depends on the whole key, not on part of it. This only bites when the key is composite. In Enrolment(StudentID, CourseID, StudentName, CourseName, Grade), the key is (StudentID, CourseID). StudentName depends only on StudentID. CourseName depends only on CourseID. Those are partial dependencies. Grade depends on the whole key and may stay. The repair is three tables: Student(StudentID, StudentName), Course(CourseID, CourseName), Result(StudentID, CourseID, Grade).

Third normal form. The table is in 2NF, and no non-key field depends on another non-key field. Student(StudentID, Name, DeptID, DeptBuilding) is in 2NF if StudentID is the only key, but DeptBuilding depends on DeptID, not directly on the student. That is a transitive dependency. The repair is Student(StudentID, Name, DeptID) and Department(DeptID, DeptBuilding).

A registration table that breaks the rules, and the same facts in third normal form.



Names and course titles are stored once. The grade stays in the link table.

After the split, a department that moves building is edited in one row. A student who changes department changes one DeptID. The grade of a course does not have to be retyped because a name was spelled wrong. That is the practical prize. You do not normalise because the phrase is impressive. You normalise so that one fact lives in one place.

EXAM TIP

When you are asked to convert a table, write the new tables with their keys underlined or named. A paragraph that says 'I would normalise it' has not converted anything.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

1. Which SQL clause keeps only some of the rows?
 - A. SELECT
 - B. FROM
 - C. WHERE
 - D. AS
2. Using the employees table in this lecture, what does `SELECT name FROM employees WHERE salary >= 80000 ORDER BY name;` return?
3. Enrolment(StudentID, CourseID, StudentName, Grade) has key (StudentID, CourseID). Which normal form does it break, and what tables replace it?

ANSWERS

1. C. WHERE filters rows. SELECT chooses columns. AS names an expression.
2. Ayesha and Hina, in that order, because both have salary at least 80000 and Ayesha comes before Hina alphabetically. Bilal is excluded.
3. It breaks second normal form because StudentName depends only on StudentID. Replace it with Student(StudentID, StudentName) and Result(StudentID, CourseID, Grade).

Lecture 3.5 Testing and debugging

OUTCOMES

Write a small test that calls a function and checks the returned value. Use a print, and describe a breakpoint, to find a wrong value.

A test is a call you expected

Class XI dry-ran code. Class XII writes the check down so it can be repeated. A **unit test** calls one function with one input and states the output. If someone later 'improves' the function and breaks that case, the test fails immediately. The test is not a proof that the function is right for every input. It is proof that the cases you cared about still hold.

```
def larger(a, b):
    if a > b:
        return a
    return b

def test_larger():
    assert larger(3, 5) == 5
    assert larger(5, 3) == 5
    assert larger(4, 4) == 4

test_larger()
print("tests passed")
```

Output

```
tests passed
```

`assert` stops the program if the condition is false. That is what you want from a test. A test that prints 'ok' even when the numbers are wrong is not a test. Include a case where the two arguments are equal, and a case where the larger one is the second argument. The interesting bug is almost never in the first example you thought of.

Seeing the middle

A failing test tells you the function is wrong. It does not tell you which line. A `print` of the arguments at the start of the function, removed after the repair, is a legitimate tool. A **breakpoint** pauses the program on a chosen line so you can look at the variables without printing all of them. A **watch** is that variable kept visible while you step. Thonny and VS Code both do this. In a written answer it is enough to say: a breakpoint pauses execution so the state can be inspected.

Debugging is a loop. Reproduce the failure with the smallest input. Predict the value at one line. Compare. Change one thing. Run the same test again. Changing three things at once, then seeing the test pass, does not tell you which change mattered.

REMEMBER

Test the value the function returns, not the sentence you meant to print. If the specification is a number, assert the number.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

- What is a unit test for?
 - Decorating the output
 - Checking one function against expected results
 - Drawing a flowchart
 - Normalising a table
- Write two asserts for a function `passed(mark)` that should return `True` only when `mark` is 50 or more. Include one failing mark.
- The test fails. What do you do before rewriting the whole function?

ANSWERS

- B. A unit test calls a unit of code and checks the result.
- `assert passed(50) is True`. `assert passed(49) is False`. A third, `passed(80) is True`, is useful but the question asked for two.
- Reproduce it with the smallest input, print or break on the line where the value first disagrees with the dry run, and change only that. Then run the same test.

Practical work

Every program in the journal should have one test written under it: the call, the expected output, and the output you actually saw.

- Define a `Student` class with `name` and `marks`. Add a method that returns the mean. Create two objects and print both means.
- Store the same three students once as a list of lists and once as a dictionary keyed by roll number. Time is not the point: show the line that retrieves one student in each design.
- Write three names to a text file, close it, read it back, and print the line count.
- Take a one-table design that repeats a department name. Split it into third normal form and write the `SELECT` that lists students of one department.

Unit 4 Data and Analysis

Class XI stopped at a straight line and the warning that correlation is not causation. Class XII asks how a model is trained, how you know it is any good, and how a formal test separates a real difference from luck. The arithmetic stays small. The meanings do not.

OUTCOMES

By the end of this unit you should be able to: Describe a train-test split and read a confusion matrix. Build a simple visualisation that answers a stated question. Write a null and an alternative hypothesis and use a p-value against a significance level.

Lecture 4.1 Learning from data, and knowing if it worked

OUTCOMES

Contrast a rule written by a person with a model fitted to examples. Explain train-test split, and compute precision from a confusion matrix.

Rules and learning

A rule-based program does what you wrote. If the rule is wrong, you can point at the line. A **machine-learning** model is adjusted so that its outputs agree with examples. The adjustment is the 'learning'. It is not understanding, and it is not a guarantee about tomorrow. The examples are the whole of its experience. If the examples are from one college, the model has learned that college.

The work has a shape. Choose the **features**, the inputs the model may see, such as hours of sleep, and refuse features it must not see, such as a photograph when the decision should not depend on appearance. Split the rows into a **training set**, which the model is fitted on, and a **test set**, which it is scored on and which it must not have been fitted on. A score computed on the training rows answers 'did it memorise?'. A score on the test rows is the one that answers 'does it handle a new row?'. Fitting on all the data and then scoring on all the data is not a test.

A confusion matrix

For a yes-or-no decision, count four numbers. **True positive**: the model said yes and the truth was yes. **False positive**: the model said yes and the truth was no. **False negative**: the model said no and the truth was yes. **True negative**: the model said no and the truth was no.

A confusion matrix. Precision uses the predicted-positive column: $90 / (90 + 30) = 0.75$.

	Predicted positive	Predicted negative
Actual positive	90	10
Actual negative	30	70

True positives sit on the top left of this layout.

In that matrix there are 90 true positives, 30 false positives, 10 false negatives, and 70 true negatives. **Precision** is the share of the model's 'yes' answers that were right: $90 / (90 + 30) = 0.75$, or 75 percent. **Recall** is the share of the real 'yes' cases that the model found: $90 / (90 + 10) = 0.90$. **Accuracy** is the share of all rows that were right: $(90 + 70) / 200 = 0.80$. These three numbers answer different complaints. A fire alarm with high precision rarely cries wolf. A fire alarm with high recall rarely misses a fire. You cannot tell from accuracy alone which failure you have.

Feature engineering means deciding which columns exist before the model runs: converting a date into 'day of week', or refusing a column that leaks the answer. It is ordinary careful preparation. It is not a separate kind of intelligence. A model cannot repair a column you should not have included, and a pretty accuracy on a leaked column is a false report.

REMEMBER

If the question gives a matrix, write the fraction before the percentage. Markers award the fraction 90/120 even when the percentage is rounded differently.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

1. Precision, for the matrix in this lecture, is
 - A. 90/100
 - B. 90/120
 - C. 160/200
 - D. 30/100
2. Why must the test rows be kept out of training?
3. A medical screen has recall 0.99 and precision 0.20. What do those two numbers mean in ordinary language?

ANSWERS

1. B. Precision is true positives divided by predicted positives, $90/(90+30)$.
2. The test is meant to imitate a new case. If the model was fitted on those rows, a high score shows memory, not the ability to handle new data.
3. It finds 99 percent of the real cases, and only 20 percent of its alarms are real cases. It rarely misses, and it often alarms when nothing is wrong.

Lecture 4.2 Visualisation that makes a claim

OUTCOMES

Choose a chart for a question and label it so the claim is checkable. Use a query or a short program to produce the numbers behind the chart.

A chart is a sentence

A visualisation earns a place when it answers a question faster and more honestly than a paragraph. 'Are IT salaries higher than HR salaries in this table?' is a question. A bar of the two averages answers it. A 3D pie of the same two numbers does not answer it better, and it makes the comparison harder. Start from the question, then the chart, then the decoration, and usually skip the decoration.

The checks from Class XI still apply. A histogram is for the shape of one numeric variable. A bar chart is for categories. A line is for time or another true sequence. A scatter plot is for two numeric variables. Title the chart with the claim or the question. Label axes with units. Say how many rows. A chart of three employees is a description of those three people.

Where the numbers come from

The numbers should be reproducible. In SQL, the average salary by department is a query, not a number typed into a slide. In Python, a few lines that group the rows are the same idea. In a spreadsheet, the formula should refer to the cells, not to a value you typed over the formula. If a reader cannot see where the bar came from, the bar is an illustration, not an analysis.

Look at published charts with the same suspicion you use on a paragraph. A vertical axis that starts at 70 makes a rise from 71 to 74 look enormous. A missing year makes a decline look like a trend. These are not advanced statistics. They are reading.

EXAMPLE

Question: in the employees table, which department has the higher average pay? IT's average is $(80000 + 100000) / 2 = 90000$. HR's average is 60000. A bar chart with two bars, titled 'Average yearly pay by department (3 employees)', axis in rupees, answers the question and admits the tiny sample.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

1. Which chart best compares the average salary of two named departments?
 - A. A pie chart with a 3D effect
 - B. A bar chart of the two averages
 - C. A histogram of department names
 - D. A line chart in alphabetical order of employees
2. Give three things a chart caption must include so that a small survey is not overclaimed.
3. Why is an axis that starts at 70 a problem when the data runs from 71 to 74?

ANSWERS

1. B. Two categories and one number each is a bar chart. Alphabetical order is not time.
2. The question that was asked, the number of rows or people, and the units. It should not generalise beyond that group.
3. The visual jump looks large because the baseline is missing. The actual change is 3 units, which a zero baseline would show as small.

Lecture 4.3 Hypothesis testing

OUTCOMES

Write a null hypothesis and an alternative hypothesis for a stated study. Compare a p-value with a significance level and say what the comparison does not prove.

A claim you can reject

A study compares something, such as a new revision method against the usual method. Before looking at the scores you write two hypotheses. The **null hypothesis, H_0** , is the cautious claim: there is no difference in the mean score. The **alternative hypothesis, H_a** , is the claim you are interested in: the new method has a higher mean, or a different mean, depending on the question. The test then asks how surprising the data would be if H_0 were true.

The **p-value** is that surprise, written as a probability. A small p-value means 'data like these would rarely appear if there were really no difference'. The **significance level**, often written alpha and often set at 0.05, is the line you drew in advance. If the p-value is smaller than alpha, you **reject H_0** . If it is not smaller, you **do not reject H_0** . You do not say ' H_0 is true'. Absence of a surprising difference is not proof that the difference is zero. The sample may have been too small to see a real but modest effect.

Alpha is also the probability of a particular mistake: rejecting H_0 when H_0 is actually true. At 0.05 you have accepted that, over many tests of true nulls, about one in twenty will be a false alarm. That is why 0.05 is a convention and not a law of nature, and why a result that just crosses the line is not a transformation of the college.

Saying it in a chart

A bar chart of the two group means, with the number of students in each group written on the chart, communicates the test better than the p-value alone. If the bars are almost equal, a tiny p-value is a reason to look for a huge sample, not a reason to announce a revolution. If the bars differ by a mark out of 100, say so. **Statistical significance** and **a difference large enough to matter** are different sentences. Write both.

EXAMPLE

H_0 : the mean Computer Science score with the new method equals the mean with the usual method. H_a : the new-method mean is higher. Alpha is 0.05. The test gives $p = 0.03$. Because 0.03 is less than 0.05, reject H_0 . There is evidence of a higher mean. If instead $p = 0.22$, do not reject H_0 . Report the two means anyway, so a reader can see the size of the gap the study failed to establish.

EXAM TIP

Write H_0 and H_a about the population means, not about the sample. The sample means are already known; there is nothing to hypothesise about them.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

1. A significance level of 0.05 is
 - A. the probability of rejecting a true null hypothesis, under the test's assumptions
 - B. proof that the alternative is true
 - C. the sample size
 - D. the mean of the data
2. A teacher tests a new drill. Write H_0 and H_a if the hope is a higher mean score.
3. Alpha is 0.05 and the p-value is 0.08. What is the decision, and what must you not claim?

ANSWERS

1. A. Alpha is the false-alarm rate you accepted in advance. It is not a sample size and it does not prove H_a .
2. H_0 : the mean score with the new drill equals the mean with the usual drill. H_a : the mean with the new drill is higher.
3. Do not reject H_0 . You must not claim that the study proved there is no difference. It failed to find a difference surprising enough at this alpha.

Practical work

A spreadsheet is enough. If the lab has Python, a notebook is also enough. Do not download a large model.

- 1 Invent 20 labelled rows: ten 'late' and ten 'on time', with hours of sleep as a number. Split them into 15 training rows and 5 test rows by a rule you write down before you look at the scores.
- 2 From a confusion matrix your teacher gives you, compute precision and recall and say which one a school would care about for a false alarm.
- 3 Write H_0 and H_a for a claim about a revision method. Sketch a bar chart that would support H_a , and one that would not.

Unit 5 Applications and Impacts of Computing

This unit turns the definitions into local design. You propose a use of IoT, cloud, or a shared ledger that would matter in Pakistan, you say what deep learning adds to the models of Unit 4, and you treat privacy, security, and unequal access as design constraints rather than as posters.

OUTCOMES

By the end of this unit you should be able to: Propose a concrete local application and the data it would need. Describe deep learning at the level of layers and features. Recommend a response to a privacy conflict and to a stated security threat.

Lecture 5.1 Designs for Pakistan: IoT, cloud, and shared records

OUTCOMES

Propose an application whose users and data you can name. Say which technology is doing which part of the job.

A design has nouns

A good answer at this grade is not the sentence 'IoT can help Pakistan'. It names a user, a sensor or a service, the decision that changes, and the data that must not leak. The technology is chosen after the problem, not before it.

- **Agriculture.** A moisture sensor in a field, a small radio link, and a phone message when the soil stays dry. The cloud holds the history so a district can see which areas are dry. The farmer does not need a blockchain for this. A database with backups is enough.

- **Flood or heat warning.** Sensors or published readings, a cloud service that sends a message, and a path that still works by SMS when a smartphone app is the wrong assumption.
- **A college service.** Cloud hosting so that a result portal survives a failed lab PC and can be reached from home. The design includes a phone layout, from Unit 1, and a paper fallback when the network fails, from the digital-divide lecture in Class XI.
- **A shared history.** Several organisations that do not share one boss, such as the stages of a food shipment, may benefit from a ledger that makes later editing visible. Name the organisations. If there is a single college administration, an ordinary database with access control is the simpler honest design.

Cloud computing fits when many people must reach one service and when the load changes, for example admissions week. It does not remove the duty to keep a backup, or the duty to say which country the data sits in if the data is sensitive. IoT fits when a physical quantity should change an action. A shared ledger fits when the problem is trust between organisations about a history. Using all three in one paragraph, for a problem that needed one, is how designs become unbuildable.

EXAM TIP

Three concrete points beat five abstract ones. 'SMS when soil moisture stays under a stated level for a day, history stored for the current season, only the farmer and the extension office can see the field's identity' is an answer.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

1. Which problem is a poor fit for a blockchain and a good fit for an ordinary database?
 - A. Several competing firms need one shipment history that none of them can quietly edit
 - B. One college library tracks loans and must correct a mistaken entry
 - C. A group of hospitals that do not trust one operator want a shared vaccination log
 - D. Companies want a record of custody for a medicine batch
2. Propose an IoT use for a school in a hot district. Include the sensor, the action, and one privacy limit.
3. Why might a result portal be hosted in the cloud rather than only on a PC in the lab?

ANSWERS

1. B. One administrator who must be able to correct errors is an ordinary database problem.
2. A temperature sensor in the classroom sends a reading each hour. If it stays above a chosen level, the office is messaged so the class can move or finish early. The system stores room numbers, not student names.
3. Students can reach it from home, it does not depend on one lab machine being switched on, and the college can add capacity during admissions without buying that capacity for the whole year.

Lecture 5.2 Deep learning, privacy, and policy

OUTCOMES

Describe deep learning as a network that builds features in layers. Propose a compromise for one data-sharing conflict.

Layers

A **neural network** is a stack of simple calculations whose internal numbers are adjusted during training. **Deep learning** means that stack has many layers. The early layers respond to small regularities, such as an edge in a picture or a short sound. Later layers combine those into larger pieces, such as a letter or a word. The important point for this course is not the formula. It is that the features are learned from data rather than all being written by a programmer, and that the network is only as representative as the data.

Applications you should be able to name honestly: reading numbers on a form, turning speech into text, translating a sentence, and spotting a crack in a photograph of a road. In each case the system can be wrong, and the cost of the

error depends on the use. A wrong translation of a menu is not a wrong reading of a medical scan. Precision and recall, from Unit 4, are how you talk about that cost.

When interests conflict

Data sharing and privacy pull apart. A hospital study needs records. A patient needs those records not to become a product. A college wants to know which videos students finish. A student wants not to be scored, secretly, on how often they pause. There is no design in which every party gets everything. A **policy** is the compromise, written down so it can be criticised.

- Collect the smallest set of fields that answers the stated question.
- Say who receives the data, and forbid a second use that was not stated.
- Keep it for a stated time and then delete it.
- Prefer totals over named rows when the question is about a class rather than a person.
- Give the person a way to see what is held, and a person to appeal to when a model affects them.

These are not decorations on an otherwise finished system. They are requirements, in the SDLC sense, and they change the tables. A design that 'will add privacy later' has decided to collect everything now.

EXAMPLE

A ride-hailing data set could be shared with a city as hourly counts of trips between areas, rather than as a file of named passengers. The city learns where buses are needed. The passenger's evening travel is not published. That is a compromise, and it should be described as one, not as the city 'not really using data'.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

1. What distinguishes deep learning from a short neural network in this course's terms?
 - A. It does not use data
 - B. It uses many layers and learns features from the data
 - C. It is always correct
 - D. It replaces the need for a test set
2. Give one application of deep learning and one way it can fail that a user would care about.
3. A researcher wants student location data 'to improve education'. Propose a narrower sharing rule.

ANSWERS

1. B. Depth means many layers that build features. It still needs data and a test, and it can be wrong.
2. Speech-to-text can produce captions. It can mis-hear a word, and a caption that changes the meaning of a safety instruction is a failure the user cares about.
3. Share counts of how many students are on campus each hour, not each student's path. Keep the counts one term. Do not allow the file to be reused for discipline or for advertising.

Lecture 5.3 Threats, controls, and equal access

OUTCOMES

Match a threat to a control: 2FA, biometrics, or encryption. Describe one concrete step toward more equal access to a service.

Threats worth naming

You are not asked to carry any of these out. You are asked to recognise them and to name a control. **Phishing** tries to talk you into giving away a password or a code. **Malware** is software that acts against the user; ransomware and spyware are kinds you already named in Class XI. A **DDoS** tries to overwhelm a service so that real users cannot reach it. A weak shared password lets one leaked site become a leaked college portal.

Control	What it improves	What it does not do
2FA	A stolen password alone cannot open the account	It does not help if the user approves a prompt they did not start, and it fails when the second factor is lost
Biometric unlock	A quick check of the person who is present	It is a poor recovery method, and a leaked template cannot be 'changed' like a password
Encryption in transit (HTTPS)	Someone on the same Wi-Fi cannot read the page	It does not mean the site itself is honest
Encryption of stored backups	A stolen disk is not a stolen spreadsheet	It does not stop a person who is already logged in
A firewall and an account lockout	Repeated guessing and some unwanted connections	It does not replace a backup when ransomware has already run

Table. Pair the control with the threat. A list of product names is not a mitigation.

Access

Equal access is a design requirement with technical pieces. A service that only works on a new phone excludes students. A service that only works in English excludes students who can do the subject in Urdu. A video with no captions excludes deaf students and also fails anyone in a noisy bus. Collaboration on a group practical has its own rules: a shared document with named accounts, no posting of the whole group's passwords, and a copy the teacher can still open if one student leaves the group.

The career point in the syllabus is plain. Computing work is done by groups, and the groups are better when more people can join them. A college society that only meets on a platform half the class cannot open has made an accidental admissions decision. Move the meeting, or open a second channel.

REMEMBER

Confidentiality, integrity, and availability are still the three headings when a question feels vague. Say which one the threat attacks, then the control.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

- Which control directly reduces the damage of a password stolen from another website, when the user reused it?
 - A faster processor
 - Two-factor authentication
 - A larger monitor
 - Sorting the password file
- Match one control to phishing and one control to someone reading traffic on café Wi-Fi.

3. Give two changes that make an online practical more equally accessible.

ANSWERS

1. B. The reused password is not enough if a second factor is required. The better long-term habit is also not to reuse it.
2. For phishing, teach people to open the site by typing the address, and use 2FA so a captured password is incomplete. For the café, HTTPS encrypts the page in transit so the café network cannot read it.
3. Accept the work from the college lab as well as from home, and provide captions or a written alternative for any video instruction.

Practical work

The practical is a one-page design, not an implementation.

- 1 Write a one-page design for one of: a flood or heat warning for a neighbourhood, a soil-moisture service for a farm, or a result portal that stays usable on a phone. Name the data, who may see it, and how long it is kept.
- 2 Take a news claim about an AI system. Say what would count as precision or recall for that system, and whose harm a false positive would be.
- 3 List the security controls on one service you use. Mark each as helping confidentiality, integrity, or availability.

Unit 6 Digital Research and Entrepreneurship

The last unit asks you to communicate a finding and to test a business idea on the smallest group that could prove you wrong. Digital literacy at this grade is the research artefact. Entrepreneurship is the minimum viable product. They use the same muscle: show the evidence, and do not claim more than it supports.

OUTCOMES

By the end of this unit you should be able to: Plan a search and an artefact that answers one research question. Distinguish a prototype from an MVP, and name the riskiest assumption a test is aimed at.

Lecture 6.1 A research artefact

OUTCOMES

Turn a topic into a question that data could answer. Publish the answer in a form a reader can check.

From a topic to a question

'Impact of computers' is a topic. It cannot be finished. 'Among 30 students in this class, how many submitted the last practical from a phone, and what stopped the others?' is a question. A research artefact answers one question. The tool can be a document, a slide, a spreadsheet chart, or a short video with a transcript. The tool is chosen after the question. A video of you reading a definition is not research.

Search in layers. Start with the precise phrase. Add a source limit, such as a government or university site, when you want a document rather than a discussion. Open more than the first result. Record the title, the author or organisation, the date, and the one fact you are taking. Two sources that copy each other are one source. Class XI's checks still apply: no author, no date, no evidence, no citation.

The shape of the artefact

- 1 The question, in one sentence.
- 2 The method: who or what you examined, and who was left out.
- 3 The result, as a number, a table, or a chart with units.
- 4 The conclusion, no stronger than the method.
- 5 The sources, enough for a reader to open the same page.

Digital tools help when they keep those five visible. A spreadsheet chart with a title does. A template full of icons does not. If you present to the class, the same five should be speakable without the decoration.

EXAMPLE

Question: do the Class XII practical instructions assume a home computer? Method: survey all 28 students present on Tuesday; 4 were absent and are not in the result. Result: 11 have no computer at home and use the lab. Conclusion: instructions that require software not installed in the lab exclude those 11. This does not estimate the whole of Sindh. The artefact is one page and one bar chart.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

1. Which search habit belongs in a serious research brief?
 - A. Using only the first link
 - B. Using several precise terms and recording the source of each fact
 - C. Preferring the claim that is most surprising
 - D. Avoiding dates because they clutter the page
2. Rewrite 'social media and students' as a question you could answer in one college.
3. Name two digital tools that could present that answer, and say what each must show.

ANSWERS

1. B. The reader must be able to open the same sources. Surprise is not a test of truth.
2. Of the students in this Computer Science section, how many used a phone to submit the last assignment, and what was the main obstacle for those who did not?
3. A spreadsheet chart, showing counts and the number of students asked, and a one-page document that states the question, the method, and the limit of the conclusion.

Lecture 6.2 From a prototype to a minimum viable product**OUTCOMES**

Distinguish a prototype from an MVP. Identify a beachhead market and the riskiest assumption, and design a small test.

Three phrases that are not synonyms

A **prototype**, from Class XI, is a learning model. Paper screens are a prototype. Nobody's problem is solved by the paper, and that is acceptable, because the paper was a question. A **minimum viable product (MVP)** is the smallest real offering that a chosen group will actually use, built to test the belief that most needs to be true. A finished app with every feature you can imagine is neither of these. It is a bet.

The **beachhead market** is the first narrow group, not 'everyone in Pakistan'. 'Class XII Computer Science students at this college who already submit work late' is a beachhead. 'All students' is a fantasy of distribution. You learn more from twenty people who share a problem than from a poster aimed at a city.

The **riskiest assumption** is the belief that, if it is false, the idea is not a business. Often it is not 'can we write the program?'. You already know a group of students can write a small program. It is 'will the beachhead user change a habit?'. For a notice service, the riskiest assumption may be that students will open it on the evening before a paper. Test that before you build chat, profiles, and a points system.

A test that can say no

An MVP test names the observation in advance. 'We will give the notice service to one section for two weeks. We will count how many open the message on the day it is sent. If fewer than half do, the assumption has failed and we will not add features until we understand why.' A test with no failure line will be talked into a success afterwards.

The MVP itself should be almost embarrassing in its smallness. A teacher posting the notice by a single messaging list the students already read, and a sheet recording who saw it, may be a better MVP than an app. If that already

fails, the app would have failed too. If it succeeds, you have earned the right to make the list less clumsy.

EXAMPLE

Idea: a paid set of solved programming drills. Beachhead: students in this class who failed the last practical. Riskiest assumption: they will complete four drills in a week if the drills are short and marked. MVP: four paper-or-PDF drills and a Saturday check, not a subscription platform. Failure line: fewer than half the invited students complete two drills. Learning is then about the habit, and the platform would have been a waste.

EXAM TIP

A five-mark answer wants the definition, then the assumption, then the test. A definition alone takes the first marks and leaves the rest.

Checkpoint

Answer these before you look at the key. The short questions are written in the style used in HSSC Section B.

1. What is the role of the riskiest assumption in an MVP?
 - A. It is the feature list
 - B. It is the belief the test is designed to support or knock down
 - C. It is the company logo
 - D. It is the final exam syllabus
2. Contrast a prototype and an MVP in two sentences, using a college notice service.
3. Choose a beachhead and a failure line for a service that lends calculators.

ANSWERS

1. B. The MVP exists to test the belief that would kill the idea if it were false.
2. A prototype is paper screens used to see whether students understand the labels. An MVP is a real notice sent for two weeks to one section, used to see whether they actually open it.
3. Beachhead: Class XI students in this college who have a mathematics paper this month. Failure line: if fewer than ten of thirty invited students borrow a calculator in the two weeks before the paper, do not buy a larger stock.

Practical work

The practical product is either a three-page research brief or a one-page MVP test plan. Do one of them properly.

- 1 Write a research question about a campus computing problem. Collect either a short survey or two written sources with author and date. Present one chart or one comparison table and a conclusion that matches the sample.
- 2 For a business idea, name the beachhead user, the riskiest assumption, the smallest product that could test it, and the observation that would make you stop.